

C37-A2

Changze Wu

October 2024

1 Q1

1.1 a

As we know about the Gauss Elimination, the k^{th} step is to eliminate the elements below the diagonal in the k^{th} column of matrix, which means every elements in k^{th} column from the row $k + 1$ to n should be 0, and the k^{th} row remain unchanged.

Assume that A be $n \times n$ matrix and $\mathcal{L}_k$ be the matrix that perform the elimination at the k^{th} step. Apart from that, we assume that $\mathcal{I}$ be a $n \times n$ identical matrix, m_k is a vector which contains the multiplier to eliminate the elements below the diagonal in the k^{th} column. e_k is the k^{th} column of identical matrix but only the k^{th} value is 1 the others are 0.

$$e_k = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

Then we do the $m_k e_k^T$:

$$m_k e_k^T = \begin{pmatrix} 0 & 0 & \dots & 0 \\ m_{k+1} & 0 & \dots & 0 \\ m_{k+2} & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots \\ m_n & 0 & \dots & 0 \end{pmatrix}$$

Which matrix has zeros everywhere except in the k^{th} column, where the elements from the $(k + 1)^{th}$ row downward are m_k . In this way, we consider $\mathcal{I} - m_k e_k^T$. The identity matrix ensures that all entries of matrix remain unchanged, except for the k^{th} column. The subtraction of $m_k e_k^T$ eliminates the entries below the diagonal in the k^{th} column by subtracting appropriate multiples of the k^{th} row.

The condition $e_i^T m_k = 0$ guarantees that only the rows below the k^{th} row are affected by the elimination process, and the upper part of the matrix remains unchanged. Thus, this condition confirms that $\mathcal{L}_k$ only affects the elements below the k^{th} diagonal.

1.2 b

$$\text{WTP} : \mathcal{L}_k^{-1} = \mathcal{J} + m_k e_k^T \quad (**)$$

By definition we can know that $\mathcal{L}_k \cdot \mathcal{L}_k^{-1} = \mathcal{J} \quad (***)$

$$\text{WTP} : (\mathcal{J} - m_k e_k^T) \cdot (\mathcal{J} + m_k e_k^T) = \mathcal{J}$$

We expand the left side, then we will get the formula:

$$\mathcal{J} \cdot \mathcal{J} + \mathcal{J} \cdot m_k e_k^T - \mathcal{J} \cdot m_k e_k^T - (m_k e_k^T) \cdot (m_k e_k^T) (*)$$

For the first element in formula, we get $\mathcal{J} \cdot \mathcal{J} = \mathcal{J}$. The sum of second part and the third part is 0. The fourth part we can get the $m_k (e_k^T m_k) e_k^T$, which from the condition, we know that $e_i^T m_k = 0$, for $i = k$. Thus, we get the fourth part is also 0. Then we get the (*) is equal to $\mathcal{J}$ which we get the same consequence with (***), which means that (**), as wanted. QED

1.3 c

From the part a, we can know that

$$\mathcal{L}_k = \mathcal{I} - m_k e_k^T \quad (1)$$

$$\mathcal{L}_j = \mathcal{I} - m_j e_j^T \quad (2)$$

When we multiply them together, we can get the:

$$\mathcal{L}_k \cdot \mathcal{L}_j = \mathcal{I} \cdot \mathcal{I} - \mathcal{I} \cdot m_k e_k^T - \mathcal{I} \cdot m_j e_j^T + (m_k e_k^T) \cdot (m_j e_j^T) = \mathcal{I} - m_k e_k^T - m_j e_j^T$$

Which from part b, we know $(m_k e_k^T) \cdot (m_j e_j^T) = 0$ for $j < k$ and we can also get that by change the sign of multiplier :

$$(\mathcal{L}_k \cdot \mathcal{L}_j)^{-1} = \mathcal{I} + m_k e_k^T + m_j e_j^T$$

then we can get

$$(\mathcal{L}_k \cdot \mathcal{L}_j)^{-1} - \mathcal{I} = m_k e_k^T + m_j e_j^T$$

as wanted. QED

1.4 d

From the part a, we can know that $\mathcal{L}_k = \mathcal{J} - m_k e_k^T (*)$

Now we consider

$$\begin{aligned}
 \tilde{\mathcal{L}}_k &= P_i \mathcal{L}_k P_i \\
 &= P_i (\mathcal{J} - m_k e_k^T) P_i && \text{by } (*) \\
 &= P_i \mathcal{J} P_i - P_i m_k e_k^T P_i \\
 &= \mathcal{J} - (P_i m_k) (e_k^T P_i)
 \end{aligned}$$

if $k \geq i$, then there might be $j = k$, then the column k will swapped with the column i , which not only swapped the row but also the column.

Because P_i is a permutation matrix that swaps rows i and j , and consequently, when post-multiplied, it swaps columns i and j .

However, we know that $k < i < j$, which means we will not swapped the column k into another position and the elements in origin matrix are all 0 except column k , thus, the operation of swap column do nothing. Thus, we can know that $(P_i m_k) (e_k^T P_i)$ is the matrix with the same structure as $m_k e_k^T$, but with the entries corresponding to rows i and j swapped. as wanted. QED

2 Q2

2.1 a

We begin with the first column. The largest absolute value in the first column is 8, which is in row 2. To avoid small pivoting elements, we swap row 1 and row 2.

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \Rightarrow PA = \begin{pmatrix} 8 & 32 & 8 \\ 1 & 8 & 13 \\ 2 & 5 & 10 \end{pmatrix}$$

Then we do the elimination :

For row 3, the multiplier is $\frac{2}{8} = 0.25$:

$$\text{Row 3} = (2 \quad 5 \quad 10) - 0.25 \times (8 \quad 32 \quad 8) = (0 \quad -3 \quad 8)$$

Same operation for row 2, the multiplier is $\frac{1}{8} = 0.125$

$$\text{Row 2} = (1 \quad 8 \quad 13) - 0.125 \times (8 \quad 32 \quad 8) = (0 \quad 4 \quad 12)$$

Then we get :

$$PA = \begin{pmatrix} 8 & 32 & 8 \\ 0 & 4 & 12 \\ 0 & -3 & 8 \end{pmatrix}$$

Same operation for row 3 again, the multiplier is $-\frac{3}{4}$:

$$\text{Row 3} = (0 \quad -3 \quad 8) - \left(-\frac{3}{4} \times (0 \quad 4 \quad 12)\right) = (0 \quad 0 \quad 17)$$

Then we get the upper triangle matrix :

$$U = \begin{pmatrix} 8 & 32 & 8 \\ 0 & 4 & 12 \\ 0 & 0 & 17 \end{pmatrix}$$

Then we construct the lower triangle :

$$L = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{8} & 1 & 0 \\ \frac{1}{4} & -\frac{3}{4} & 1 \end{pmatrix}$$

2.2 b

First we get

$$Pb = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 7 \\ -16 \\ 6 \end{pmatrix} = \begin{pmatrix} -16 \\ 6 \\ 7 \end{pmatrix}$$

Then we solve $Ld = b$ for d :

$$\begin{pmatrix} 1 & 0 & 0 \\ 0.125 & 1 & 0 \\ 0.25 & -\frac{3}{4} & 1 \end{pmatrix} \begin{pmatrix} d_1 \\ d_2 \\ d_3 \end{pmatrix} = \begin{pmatrix} -16 \\ 6 \\ 7 \end{pmatrix}$$

From that we get $d_1 = -16$, $d_2 + 0.125d_1 = 6$, $d_3 + 0.25d_1 + \frac{-3}{4}d_2 = 7$. Then we get $d = (d_1 \ d_2 \ d_3) = (-16 \ 8 \ 17)$.

$$d = \begin{pmatrix} -16 \\ 8 \\ 17 \end{pmatrix}.$$

Then we solve the $Ux = d$:

$$\begin{pmatrix} 8 & 32 & 8 \\ 0 & 4 & 12 \\ 0 & 0 & 17 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} -16 \\ 8 \\ 17 \end{pmatrix}$$

Same with solve the d , we can get the

$$x = \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}$$

2.3 c

Gaussian elimination is often implemented in the form $PA = LU$ because it has several advantages. First, the LU factorization can be reused to efficiently solve multiple right-hand sides without having to redo the elimination, making the computations faster. Second, the use of a permutation matrix P ensures numerical stability because rotations prevent division by small numbers. Finally, solving the system using LU involves efficient forward and backward substitution steps that are faster than performing full elimination each time. This method is stable and efficient, especially suitable for large systems or repeated solutions.

3 Q3

3.1 a

The general structure of an $n \times n$ matrix just before the k^{th} stage of the elimination,

$$A = \begin{pmatrix} * & * & * & \cdots & * & \cdots & * \\ 0 & * & * & \cdots & * & \cdots & * \\ 0 & 0 & \ddots & \cdots & \ddots & \cdots & \ddots \\ 0 & 0 & 0 & A_{k,k} & A_{k,k+1} & \cdots & A_{k,n} \\ 0 & 0 & 0 & A_{k+1,k} & A_{k+1,k+1} & \cdots & A_{k+1,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & A_{n,k} & A_{n,k+1} & \cdots & A_{n,n} \end{pmatrix}$$

1. Row change : Suppose the largest element is located in row i , where $i \geq k$, then we should find $P_i A = A'$, which $A' =$

$$A' = \begin{pmatrix} * & * & * & \cdots & * & \cdots & * \\ 0 & * & * & \cdots & * & \cdots & * \\ 0 & 0 & \ddots & \cdots & \ddots & \cdots & \ddots \\ 0 & 0 & 0 & A_{i,k} & A_{i,k+1} & \cdots & A_{i,n} \\ 0 & 0 & 0 & A_{k+1,k} & A_{k+1,k+1} & \cdots & A_{k+1,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & A_{n,k} & A_{n,k+1} & \cdots & A_{n,n} \end{pmatrix}$$

2. Column change : Suppose the largest element in the row is located in column j , where $j \geq k$, then we should find P_j for $AP_j = A''$, with $A'' =$

$$A'' = \begin{pmatrix} * & * & * & \cdots & * & \cdots & * \\ 0 & * & * & \cdots & * & \cdots & * \\ 0 & 0 & \ddots & \cdots & \ddots & \cdots & \ddots \\ 0 & 0 & 0 & A_{i,j} & A_{i,k+1} & \cdots & A_{i,n} \\ 0 & 0 & 0 & A_{k+1,j} & A_{k+1,k+1} & \cdots & A_{k+1,n} \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & A_{n,j} & A_{n,k+1} & \cdots & A_{n,n} \end{pmatrix}$$

3.2 b

First we multiply P, we get $PAx = Pb$, then we consider the given condition $PAQ = LU$, for we need to make A don't change the column, we should multiply Q^T each side to make A don't change the column. Thus, we get $PAQQ^T = PA = LUQ^T$, then we substitute it into, we get the $LUQ^T x = Pb$

Then we assume 1. $y = Q^T x$, 2. $d = Uy$, 3. $Ld = Pb$

Then we solve the 3 first for d, and 2 for y and 1 for x.

Q's effect :

- Q reorders the columns of A, affecting the order of the unknown variables.
- It ensures numerical stability by enabling full pivoting, which minimizes the risk of small pivots.

- After solving the system, Q^T is applied to recover the original variable order.

3.3 c

First we find 32 is the biggest one in matrix, than we do the operation

$$PAQ = \begin{pmatrix} 32 & 8 & 8 \\ 8 & 13 & 1 \\ 5 & 10 & 2 \end{pmatrix}$$

Then we do some eliminations, for row 3, the multiplier is $\frac{5}{32}$

$$\text{Row 2} = (5 \ 10 \ 2) - \frac{5}{32} \times (32 \ 8 \ 8) = (0 \ 8.75 \ 0.75)$$

For row 2, the multiplier is $\frac{8}{32} = 0.25$.

$$\text{Row 3} = (8 \ 13 \ 1) - 0.25 \times (32 \ 8 \ 8) = (0 \ 11 \ -1)$$

Full pivoting again, we find that the biggest one is 11:

$$PAQ = \begin{pmatrix} 32 & 8 & 8 \\ 0 & 11 & -1 \\ 0 & 8.75 & 0.75 \end{pmatrix}$$

Continue elimination, the multiplier is $\frac{8.75}{11} = \frac{35}{44}$ then we will get the

$$U = \begin{pmatrix} 32 & 8 & 8 \\ 0 & 11 & -1 \\ 0 & 0 & \frac{17}{11} \end{pmatrix}$$

the corresponding lower matrix is

$$L = \begin{pmatrix} 1 & 0 & 0 \\ \frac{8}{32} & 1 & 0 \\ \frac{5}{32} & \frac{35}{44} & 1 \end{pmatrix}$$

$$P = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

$$Q = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

We calculate

$$Pb = \begin{pmatrix} -16 \\ 6 \\ 7 \end{pmatrix}$$

First we solve $Ld = Pb$ for d

$$\begin{pmatrix} 1 & 0 & 0 \\ \frac{8}{32} & 1 & 0 \\ \frac{5}{32} & \frac{35}{44} & 1 \end{pmatrix} \begin{pmatrix} d_1 \\ d_2 \\ d_3 \end{pmatrix} = \begin{pmatrix} -16 \\ 6 \\ 7 \end{pmatrix}$$

Then we get the

$$d = \begin{pmatrix} -16 \\ 10 \\ \frac{17}{11} \end{pmatrix}$$

Second we solve $Uy = d$

$$\begin{pmatrix} 32 & 8 & 8 \\ 0 & 11 & -1 \\ 0 & 0 & \frac{17}{11} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} -16 \\ 10 \\ \frac{17}{11} \end{pmatrix}$$

Then we get the

$$y = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}$$

Third we solve $Q^T x = y$ for x :

$$\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}$$

We get the final answer

$$x = \begin{pmatrix} 1 \\ -1 \\ 1 \end{pmatrix}$$

4 Q4

4.1 a

First we should compute the LU Factorization, this process will cost the $\frac{n^3}{3} + O(n^2)$ which is $O(n^3)$.

Then we have the factorization of $PA = LU$ and solve several systems such as $Ax_i = d_i$, we only need to solve with the same coefficient matrix A but different right hand sides. 1) Solve $Ly_i = Pt_i$ (forward substitution) with $O(n^2)$ flops. 2) Solve $Ux_i = y_i$ (backward substitution) with $O(n^2)$ flops.

In the meanwhile, we know that we will computing Forward / Backward for $O(n^2)$ total. And there are k systems in total.

Thus, we will get the $O(n^3) + k \cdot O(n^2)$. This is efficient because the expensive factorization is done only once, and the solutions of the individual systems are much cheaper.

4.2 b

For $k = n$, than we choose t_i be the e_i , which is the each column vector of the $n \times n$ of I identical matrix. Then we calculate x_i which all are the column vectors of A^{-1} .

Then we should calculate each:

$$Ax_1 = e_1, Ax_2 = e_2, \dots, Ax_n = e_n$$

We follow the same process in (a) and solve for each x_i then combine them together to $(x_1, x_2, \dots, x_n)$

Thus, the complexity of these process is LU Factorization + backward / forward substitution which cost is $O(n^3) + n \cdot O(n^2) = O(n^3)$

4.3 c

Method 1: Compute A^{-1} then $x = A^{-1}b$

From part b, we know that we calculate the A^{-1} cost $O(n^3)$. Then we compute the $x = A^{-1}b$ using matrix-vector multiplication, which cost $O(n^2)$. because multiplying a $n \times n$ matrix by a vector requires n^2 operations.

In summary, this method will cost $O(n^3)$

Method 2: Gaussian Elimination with $PA = LU$

The cost of performing LU factorization with partial pivoting is $O(n^3)$, as it involves a series of elimination steps to reduce the matrix to upper and lower triangular forms. And for Forward / Backward substitution will cost total $O(n^2)$

In summary, method 2 also cost $O(n^3)$

While both methods have the same asymptotic complexity $O(n^3)$, Gaussian elimination with $PA = LU$ factorization is better. It is numerically more stable, requires less storage, and avoids the unnecessary computation of the inverse matrix A^{-1} . Therefore, in most practical cases, Gaussian elimination is the better choice.

4.4 d

The complexity of Cramer's rule article

According to the article about Cramer's rule, we can do below steps:

1. First we should do the LU factorization to compute the determinant of A . When we do the factorization we cost $O(n^3)$ and then we compute the $\det(A) = u_{11} \cdot u_{22} \cdot \dots \cdot u_{nn}$ will cost the $O(n)$. Thus, the sum of this is $O(n^3)$

2. One for each matrix A_i , which is the matrix obtained by replacing one column of A with a vector. Since there are n such matrices, computing each of their determinants also requires $O(n^3)$ flops. Thus the total complexity is $n \cdot O(n^3) = O(n^4)$.

In summary, we conclude that when we use Cramer's rule, we will cost $O(n^3) + O(n^4) = O(n^4)$

According to the part c, we know that when we use Gaussian Elimination algorithm with $PA = LU$ factorization only cost $O(n^3)$, which is better than the Cramer's rule.